

テクニカルリファレンス

TDS Toolkit プロトコルデコーダの作成

TDS 500、TDS 600、TDS 700
シリーズのデジタイジングオシロスコープ用

文書 TDSTK-DEC-01
Copyright © 2026 Jared Cabot. MIT ライセンス。

TDSTK-DEC-01

Copyright © 2026 Jared Cabot. 無断複製・転載を禁じます。

本書は TDS Toolkit のデコーダプラグインインターフェースについて説明します。本書の情報は、これまでに公開されたすべての資料に優先します。

TDS Toolkit はホストソフトウェアであり、Tektronix の製品ではありません。Tektronix および TDS は、本ソフトウェアが対応する計測器を指すためにのみ使用しています。

本ソフトウェアは MIT ライセンスの下で公開されています。本ソフトウェアは、明示または黙示を問わず、いかなる種類の保証もなく「現状のまま」提供されます。全文は LICENSE としてプログラムに付属します。

目次

	関連マニュアル	iv
1	デコーダのモデル	1-1
	デコーダとは	1-1
	1 ファイル、1 プロトコル	1-1
	コントラクト	1-1
	検出	1-1
	ファイルの所在	1-1
2	Signal	2-1
	デコードに渡されるもの	2-1
	サンプルとタイミング	2-1
	サンプルではなくレベル	2-1
	Signal の生成元	2-2
3	デコーダのコントラクト	3-1
	クラス属性	3-1
	4 つの属性	3-1
	役割とソース	3-1
	メソッド	3-2
	decode	3-2
	パラメータ	3-3
	Param 型	3-3
4	フレームと描画	4-1
	Frame	4-1
	1 つのデコード済みイベント	4-1
	ステータス	4-2
	結果の描画	4-2
	4 つの小さなメソッド	4-2
	色	4-2
5	デコーダの構築	5-1
	実例	5-1
	プロトコル	5-1

手順	5-1
コード	5-1
インポートと属性	5-1
ビットの復元	5-2
ワードのまとめ	5-2
描画	5-3
シリアル近道	5-4
ビットサンプリングを再実装しない	5-4
6 インストールとテスト	6-1
インストール	6-1
置いて開く	6-1
テスト	6-1
合成した Signal で動かす	6-1
すべてのデコーダをテストする	6-2
チェックリスト	6-3
頼る前に	6-3

表一覧

表 1-1: デコーダサブシステムのファイル	1-2
表 2-1: Signal のメンバー	2-2
表 3-1: Decoder のクラス属性	3-1
表 3-2: Decoder のメソッド	3-2
表 3-3: Param の種別	3-3
表 4-1: Frame のフィールド	4-1
表 4-2: Frame のステータスビット	4-2

序文

本マニュアルでは、TDS Toolkit にプロトコルデコーダを追加する方法を説明します。デコーダは1つの Python ファイルです。取り込んだ波形を読み取り、デコードしたイベントのリストを返します。プログラムはそれを一覧に表示し、入力を割り当て、実行し、追加のコードなしで結果を描画します。

読者は Python に慣れた現役のプログラマーであることを前提とします。プログラムのそれ以外の部分に関する知識は不要です。インターフェース全体は、1つの基底クラスと3つの小さなデータ型であり、いずれも `tds_decode.py` モジュールにあります。

本マニュアルは、プログラムに付属する2つのファイル、すなわちコピー元のひな形である `decoders/_template.py` と、以降の内容を簡潔にまとめた `decoders/README.txt` を参照しながらお読みください。

関連マニュアル

インターフェースと、本マニュアルと併せて読むべきファイル:

- `decoders/_template.py` - コピー元のひな形。
- `decoders/README.txt` - 以降の内容の簡潔版。
- `tds_decode.py` - エンジン。Decoder 基底クラスと `Signal`、`Frame`、`Param` の各型。

デコーダのモデル

デコーダとは

1 ファイル、1 プロトコル	デコーダは、decoders フォルダ内にある 1 つの Python ファイルで、Decoder のサブクラスを 1 つ定義します。1 ファイルが 1 プロトコルです。波形タブは見つかったすべてのデコーダを一覧表示するため、プロトコルの追加はファイル 1 つで済み、プログラムのそれ以外は何も変わりません。 デコードは、波形タブが既に保持している取り込み済みの波形、すなわち計測器から読み取ったチャンネル、またはファイルから読み込んだ保存済みリファレンスに対してのみ実行されます。デコーダは計測器に一切触れません。それがこの設計の要点です。デコードはホスト側で、取り込み後のサンプルに対して行われるため、バスデコードアプリケーションを自前で実行できないファームウェアのベースモデルを含め、あらゆる TDS で動作します。
コントラクト	インターフェース全体は Decoder 基底クラスです。デコーダは 4 つのクラス属性を設定し、1 つのメソッドを実装します。残る 3 つのメソッドには実用的な既定の実装があり、そのプロトコル固有のフィールドをテーブルやグラフに描画する場合にのみオーバーライドします。デコーダが扱う 3 つのデータ型 Signal、Frame、Param については、以降の章で説明します。 起動時、および波形タブを開くたびに、プログラムは decoders フォルダ内の *.py ファイルを走査し、それぞれを個別にインポートして、見つかった Decoder のサブクラスをすべてインスタンス化します。名前がアンダースコアで始まるファイルはヘルパーとして扱われ、スキップされます。これにより _template.py はひな形のままとなり、空のプロトコルとして一覧に現れることはありません。
検出	インポートは防御的です。構文エラーであれ不正な参照であれ、インポートできないファイルは、一覧全体を巻き添えにするのではなく、プログラムログに記録を残したうえでスキップされます。作りかけのデコーダが、動作するデコーダを隠してしまうことはありません。2 つのフォルダが同じ NAME のデコーダを提供する場合は後のフォルダが優先されるため、ローカルで書いたコピーが付属のものを上書きします。

ファイルの所在

プログラムに付属するデコーダは、インストール先内の decoders フォルダーにあります。書き込み可能なコピーが設定ファイル `tdstoolkit.json` のそばに保持され、マスクライブラリとまったく同じように、実行のたびに付属セットから補充されます。後のビルドで追加されたデコーダは、ローカルで書いたものを上書きせずにコピーされます。付属デコーダを削除しても、付属セットは常に存在すべきものなので、次の実行時に復元されます。新しいファイルを書き込み可能なフォルダーに置けば、次にタブを開いたときに一覧へ現れます。リビルドは不要です。

表 1-1: デコーダサブシステムのファイル

ファイル	役割
<code>tds_decode.py</code>	エンジン。Decoder 基底クラス、Signal、Frame、Param の各型、ステータス定数、共有のシリアルヘルパー、および検出処理。デコーダはここからインポートし、これを変更することはありません。
<code>decoders/_template.py</code>	ひな形。これをコピーして新しいデコーダを作り始めます。
<code>decoders/README.txt</code>	簡潔版の手引きと、サンプルデコーダの一覧。
<code>decoders/.py</code>	それぞれ 1 プロトコル。 <code>uart.py</code> 、 <code>i2c.py</code> 、 <code>spi.py</code> 、 <code>can.py</code> など。追加しようとするプロトコルに最も近いものを読んでください。

Signal

デコードに渡されるもの

	decode() メソッドには、プロトコルが必要とする入力ごとに1つの Signal が渡されます。Signal は、取り込んだ1つのレコードを、デコーダが必要とするもの、すなわち整数のサンプルと、サンプルのインデックスを時刻に変換するタイミング情報だけに絞ったものです。電圧値や表示スケールは持ちません。デコーダはサンプルをしきい値と比較するだけで電圧を知る必要はないため、生のデジタイザカウントはスケールリングされないまま渡され、25
サンプルとタイミング	万点のレコードでも受け渡しのコストはかかりません。判定しきい値、ヒステリシスの不感帯、および信号のスパンは、decode() が呼び出される前に計算されます。デコーダが独自にしきい値を計算することはありません。しきい値はレコードの最大値と最小値の midpoint であり、ヒステリシスはスパンの 8 分の 1 で、エッジのリングングを排除できる程度に広く、かつ緩やかなエッジがしきい値を越え損ねない程度になっています。これらの値は設定するものではなく、読み取るためのものです。
サンプルではなくレベル	ほとんどのプロトコルにとって、レコードの有用な形は生のサンプルではなく、サンプルごとの明確な論理レベルです。digitize() メソッドはまさにそれを返します。すなわち、サンプルごとに 0 または 1 を返し、レコード自身のヒステリシスを適用することで、エッジのリングングが複数の交差として読まれないようにします。一度呼び出し、その結果を順にたどってください。2つのしきい値を持つバイポーラ線など、生のカウントを必要とするプロトコルは、代わりに sig.data を直接読み取ります。

表 2-1: Signal のメンバー

メンバー	意味
sig.data	整数のサンプル。生のデジタイザカウントの、インデックス指定可能なシーケンス。
sig.n	サンプル数。len(sig.data) と同じ。
sig.dt	1 サンプルあたりの秒数。
sig.t0	最初のサンプルの時刻。トリガを基準とした秒数。
sig.first_index	最初のサンプルのレコードオフセット。ローカルのサンプルインデックスにこれを加えると、絶対的なレコード位置になります。
sig.threshold	判定レベル。単位はカウント。フレームワークが設定します。
sig.hysteresis	しきい値まわりの不感帯。単位はカウント。フレームワークが設定します。
sig.span	レコードの最大サンプルと最小サンプルの差。フレームワークが設定します。スパンがほぼ 0 の場合は、平坦で信号のないソースを意味します。
sig.time_at(i)	ローカルのサンプルインデックス i における秒単位の時刻。すなわち $t_0 + i \times dt$ 。
sig.digitize()	サンプルごとに 1 つの 0/1 レベルを持つ、新しい bytearray。ヒステリシス適用済み。
sig.auto_threshold()	threshold、hysteresis、span を再計算します。フレームワークが既に呼び出しているため、手作業で構築した Signal に対してのみ呼び出します(第 6 章を参照)。

注記. Signal

は電圧ではなく生のカウントを保持します。電圧ではなく、必ず sig.threshold と比較してください。しきい値はレコードに追従して変化するため、ある垂直スケールで正しく読めるデコーダは、他のあらゆるスケールでも正しく読めます。

Signal の生成元 decode() が受け取る Signal
は、デコーダではなくプログラムが、クラスメソッド
Signal.from_waveform() を用いて、取り込んだ波形から構築します。波形
のプリアンプルから整数レベル、サンプル間隔、トリガオフセットを読
み取って Signal を構築し、デコーダに渡す前に auto_threshold()
を呼び出します。Signal
を手作業で構築するのはテストを書くときだけで、これは第 6
章で扱います。

デコーダのコントラクト

クラス属性

デコーダのサブクラスは4つのクラス属性を設定します。NAME と DESCRIPTION は、プロトコル一覧での名称を与えます。SOURCES は、必要とする波形入力を宣言します。PARAMS は、タブが提供する設定を宣言します。それ以外はすべて継承されます。

表 3-1: Decoder のクラス属性

属性	意味
NAME	プロトコル名。一覧に表示され、グラフのラベルのタイトルとして使われます。空、または "?" のままにすると、ローダーはそのクラスを一覧に表示しません。ひな形はこの方法で一覧から除外されています。
DESCRIPTION	一覧の 2 列目に表示する 1 行の説明。
SOURCES	順序付けされた役割名のリスト。役割が 1 つなら 1 線式プロトコルであり、2 つ以上ある場合は、それぞれに取り込んだソースが割り当てられます。末尾の "?" は、割り当てなしにできる省略可能な役割を示します。
PARAMS	Param オブジェクトのリスト。1 つにつき 1 つのコントロールになります。空でもかまいません。
COLOUR	正常なフレームのシェブロンの色。"#rrggbb" 形式。色はデコーダ側で決め、プログラムにはデコーダ用の色設定はありません。プロトコル全体を 1 色にするために設定します。
FAULT_COLOUR	異常のあるフレームのシェブロンの色。既定は赤で、どのプロトコルの色の上でもエラーが目立つようにしてあります。

役割とソース

SOURCES
は、プロトコルが必要とする波形の数と、その名前の両方を表します。1線式プロトコルは、慣例として "Line" という役割を 1 つ宣言します。I2C のような 2 線式プロトコルは ["SDA", "SCL"] を宣言します。タブは役割ごとに 1 つのソース選択欄を表示し、decode() はそれぞれに対応する Signal を受け取ります。SPI の ["CLK", "MOSI", "MISO?", "CS?"] のように末尾に疑問符が付くと、オペレーターが割り当てなしにできる役割であることを示します。その場合、その Signal は decode() が受け取る辞書に存在しないため、decode() のコードでその有無を確認します。

メソッド

decode decode(self, signals, params) は、唯一の必須メソッドです。signals は、末尾の "?" を除いた各役割名を、その Signal に対応付ける辞書です。割り当てられなかった省略可能な役割は存在しません。params は、各 Param のキーを、オペレーターが選んだ値に対応付ける辞書です。このメソッドは、Frame オブジェクトのリストと、1 行のノート文字列という 2 要素のタプルを返します。

ノートは結果テーブルの上に表示されます。検出したビットレート、1 ビットあたりのサンプル数、見つかったフレーム数など、デコードが推定した内容を報告するために使います。レコードを読み取れない場合は、その理由を伝えるために使います。失敗した場合は、空のフレームリストと、問題を 1 語句で説明するノートを返してください。

表 3-2: Decoder のメソッド

メソッド	戻り値
decode(signals, params)	(frames, note)。デコード本体。必須。
columns()	Time 列より後のテーブル見出し。(見出し, 幅ピクセル数) の組のリスト。プロトコルのフィールド名を付けるためにオーバーライドします。
row(frame, ascii)	1 フレーム分のテーブルセル。時刻より後の部分が columns() と対応します。プロトコルのフィールドを整形するためにオーバーライドします。
label(frame, ascii)	グラフ上のシェブロンに使う、最も短く有用な文字列。簡潔に保つためにオーバーライドします。
colour(frame)	1 フレーム分のシェブロンの色。既定では、正常なフレームには COLOUR、異常のあるフレームには FAULT_COLOUR を用います。frame.kind で分岐してシェブロンごとに色を付けるにはオーバーライドします。

row() と label() に渡される ascii 引数は、タブの Hex / ASCII
トグルの状態です。バイトを文字として表示するか 2 桁の 16 進数として
表示するかなど、意味のある箇所ではこれに従い、意味のない箇所では
無視します。

パラメータ

PARAMS リスト内の各 Param は、タブ上の 1 つのコントロールになります。
これにより、プロトコルはユーザーインターフェースのコードなし
に独自のオプションを追加できます。コンストラクタは Param(key, label,
kind, choices, default, lo, hi, unit, note) です。key は decode()
が読み取る辞書のキー、label
はオペレーターに見えるキャプション、kind は 3
つの文字列のいずれかです。

表 3-3: Param の種別

kind	コントロールとフィールド
"choice"	ドロップダウン。choices は (値, ラベル) の組のリストで、値が params に届き、ラベルが表示されます。default は値のいずれかです。
"int"	整数の入力欄。lo と hi が範囲を制限します。unit は "bits" などのキャプションの接尾辞です。既定値 0 は慣例として「自動」を意味するものと解釈され、その旨は note に記載すべきです。
"bool"	チェックボックス。default は True または False です。

Param 型 default を省略した場合は、choice では最初の選択肢、bool では False、int
では 0 が既定となります。省略可能な note
は、ヒントとしてコントロールの横に表示されます。

フレームと描画

Frame

1つのデコード済みイベント

Frame は 1 つのデコード済みイベントです。UART の 1 文字、I2C の 1 バイト、SPI の 1 ワード、バスの状態など。この型はフラットで、すべてのプロトコルで共通です。主データ、省略可能な副データ、長さ、ステータス、種別を保持します。フレームを生成したデコーダだけがそれを描画するため、各フィールドの意味はデコーダに委ねられますが、その構造は固定です。

Frame はキーワード引数で構築します。すべてのフィールドに既定値があります。2 つのインデックスフィールドが最も重要で、最も間違いやすい箇所です。index と end_index は絶対的なレコードのサンプル位置であり、グラフはこの一方から他方までシェブロンをの枠を描きます。時刻がローカルインデックスの sig.time_at() から得られるのとまったく同じように、これらは sig.first_index にローカルのサンプルインデックスを加えて求めます。

表 4-1: Frame のフィールド

フィールド	意味
time	イベントの開始時刻。単位は秒。sig.time_at(local_index) から得られます。
index	絶対的な開始サンプル。sig.first_index + ローカルの開始位置。
end_index	絶対的な終了サンプル。sig.first_index + ローカルの終了位置。枠は index から end_index までにわたります。
value	主データ。バイト、ワード、測定値など。
tag	省略可能な副データ。I2C のアドレス、SPI の MISO ワード、測定した周波数など。未使用の場合は 0。
length	データの長さ。プロトコルが数える単位で表します。既定値は 1。
status	ステータス定数のビット単位 OR。フレームが正常な場合は OK(ゼロ)。
kind	"byte"、"addr"、"data"、"start"、"stop" などの自由な文字列。row() と label() がこれで分岐し、フレームの各部分をそれぞれ独自に描画します。

ステータス status はビットマスクです。正常なフレームは OK であり、これはゼロです。問題のあるフレームは、表 4-2 の定数を 1 つ以上 OR で組み合わせます。フレームワークは、status が 0 以外のフレームをすべてエラーとして、赤いシェブロンとラベル先頭の感嘆符で描画します。デコーダはビットを設定する以外に何もする必要はありません。

表 4-2: Frame のステータスビット

定数	値	意味
OK	0	正常。エラーなし。
FRAMING_ERROR	1	マークでなかった UART のストップビットなど、フレームがその形式の要求どおりに終了しなかった。
PARITY_ERROR	2	パリティビットがデータと一致しなかった。
TRUNCATED	4	フレームの途中でレコードが終了した。
ACK_ERROR	8	アクノリッジを期待していたバイトがアクノリッジされなかった。

使用する定数は、Decoder、Frame、Param とともに tds_decode からインポートしてください。これら以外のステータス値を定義してはいけません。レンダラーが理解するのは上記の 5 つだけです。

結果の描画

4つの小さなメソッド

基底クラスは、フレームを単純なバイトとして描画します。すなわち 16 進数、ASCII、ステータスの文字列です。代わりにプロトコル固有の用語で表示するには、columns()、row()、label() をオーバーライドします。columns() は Time 列より後の見出しを、それぞれのピクセル幅とともに返します。row() は、同じ順序で見出しごとに 1 セルを返します。label() は、波形自体に描かれるシェブロンに使う、最も短く有用な文字列を返します。

異なる種別のフレームを持つプロトコルは、これらのメソッド内で frame.kind により分岐します。これにより、START、アドレスバイト、データバイトが、テーブル上でもグラフ上でもそれぞれ区別して表示されます。label() は簡潔に保ってください。トレース上の小さな枠内に描かれるため、1~3 文字が適切な長さです。

色 4つ目のメソッド `colour(frame)` はシェブロンの色を設定し、色は完全にデコーダ側で決まります。プログラムにはデコーダ用の色設定はありません。既定では、正常なフレームをクラスの `COLOUR` で、異常のあるフレームを `FAULT_COLOUR` で塗るため、`COLOUR` を設定すればプロトコル全体が1色になります。`colour()` をオーバーライドして `frame.kind` で分岐すれば、シェブロンごとに色を塗ることができ、1本の線上のアドレス、データ、フレーミングがそれぞれ独自の色で読めます。"`#rrggbb`"を返してください。デコーダ自身が色を付けない種別については、基底クラスに委ねます。

デコーダの構築

実例

プロトコル 本章では、ひな形から完全なデコーダを 1 つ構築します。例に用いるプロトコルは、1 パルスにつき 1 ビットを運ぶ、自己クロック方式の 1 本の線です。線が立ち上がり、ビット周期の一部の時間だけハイを保ち、そして立ち下がります。ハイの時間が周期の半分より長いかどうかで、短いハイを 0、長いハイを 1 と判定します。ビットは、選択したサイズとビット順でワードにまとめられます。これは WS2812 のピクセル線や NEC 赤外線リモコンの系統であり、組み込みプロトコルには含まれていないため、既存のものの複製ではなく、正真正銘の新しいデコーダとなります。

- 手順**
1. ひな形をコピーする: `decoders/_template.py` を `decoders/pulsebits.py` にコピーします。
 2. プロトコルに名前を付ける:
クラス名を変更し、`NAME`、`DESCRIPTION`、`SOURCES` を設定します。
 3. 設定を宣言する: `PARAMS`
に、ワードサイズとビット順のコントロールを記述します。
 4. デコードを書く:
レベルの列からビットを復元し、それらをワードにまとめます。
 5. フィールドを描画する: `columns()`、`row()`、`label()`
をオーバーライドして、ワードを 16 進数と 2 進数で表示します。
 6. テストする: 第 6 章に示すとおり、合成した `Signal`
でデコーダを動かし、結果をアサートします。

コード

インポートと属性

ヘッダー部では、`tds_decode` からのみインポートし、クラス名を付け、タブが提供する 2 つの入力、すなわちワードサイズとビット順を宣言します。`SOURCES` は役割を 1 つだけ保持するため、タブには 1 つのソース選択欄が表示されます。

```

from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]

```

ビットの復元

decode() は 1 つの Signal を受け取り、平坦なソースでないことを確認し、digitize() でレコードをサンプルごとの 0/1 レベルに変換します。続いて、そのレベルの列をたどります。各ビットは立ち上がりエッジで始まります。ハイのサンプルの連続がパルスであり、それに続くローのサンプルの連続が次の立ち上がりエッジまで続き、この 2 つが合わさってビット周期となります。ハイの時間を周期と比較してビットを判定します。

```

def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = [] # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]: # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]: # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1

```

ワードのまとめ 続いてビットは、パラメータの指定に従って最上位ビットまたは最下位ビットを先頭として、word_bits 個ずつのグループにまとめられます。レコードが終了したために不足したグループも、そのまま出力されますが、レンダラーがエラーとして描画するよう、status に TRUNCATED を付けます。各 Frame は、最初のビットから時刻を取り、最初のビットの開始から最後のビットの終了までにわたります。ノートには合計が報告されます。

```
frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))
```

描画 3つの描画メソッドは、ワードを16進数と、そのビット幅の2進数で表示し、切り詰められたワードには印を付けます。label() は2桁の16進数にとどめ、エラーの場合は先頭に感嘆符を付けます。これは組み込みデコーダが従っている慣例です。

```
def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "%0db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text
```

このファイルは、完全で一覧に表示可能なデコーダです。decoders フォルダに置くと、ワードサイズの入力欄とビット順のドロップダウンを備えた "Pulse-width bits" としてタブに現れ、その唯一の入力に割り当てられた任意のソースをデコードします。

シリアル近道

ビットサンプリングを
再実装しない

実例では、そのプロトコルが自己クロック方式で組み込みのどれとも異なるため、ビットのサンプリングを自前でを行っています。RS-232、RS-422/485、MODBUS、MIDI、LIN、XBee、DMX512 など、非同期シリアルシステムのものについては、ビットサンプリングを再実装しないでください。共有され、コーパスで検証済みのヘルパー `serial_bytes()` を呼び出してください。これは、自動ボーレート検出、ビット中央でのサンプリング、パリティおよびストップビットのチェックを、実証済みの唯一の方法で行います。そして、それが返すバイトストリームの上にフレーミングを構築します。

```
from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))
```

`serial_bytes(sig, baud, framing, inverted)` は 4 つの値、すなわちバイトのフレーム、成功時は空となるエラー文字列、検出したボーレート、および 1 ビットあたりのサンプル数を返します。baud を 0 にすると、レートの検出を要求します。inverted を True にすると、本来の RS-232 のような、アイドル時ロー・アクティブ時ハイの電氣的極性として読みます。MODBUS や LIN のような上位のプロトコルは、生のバイトを得るために `serial_bytes` を呼び出し、それらを自身のフレームにグループ化して検査します。そのパターンについては `modbus.py` と `lin.py` を読んでください。

インストールとテスト

インストール

置いて開く

完成したファイルを、tdstoolkit.json のそばにある書き込み可能な decoders フォルダにコピーし、波形タブを開く、または開き直します。デコーダがプロトコル一覧に現れます。登録の手順もリビルドもありません。取り込んだソースを各役割に割り当て、パラメータを設定して実行します。

注記. デコーダが現れない場合は、インポートされなかったということです。ファイル名と例外を記した行がないか、プログラムログを確認してください。ローダーは、一覧全体を失敗させるのではなく、インポートできないファイルをスキップするため、壊れたデコーダは何も表示されずに失敗します。

テスト

合成した Signal
で動かす

デコーダは、その Signal とパラメータの純粋な関数であるため、構築したサンプル配列から Signal を作り、decode() を呼び出して結果をアサートすることで、計測器なしにテストできます。意味の分かるようにサンプルを構築し、それを Signal に包み、フレームワークが通常行う唯一の手順である auto_threshold() を呼び出して、フレームを確認します。

```

from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20          # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20          # idle low tail
    sig = Signal(stream, 1e-6)       # 1 us per sample
    sig.auto_threshold()             # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()

```

すべてのデコーダをテストする

組み込みデコーダはすべて、この方法で検査されています。RS-232 デコーダは実際の取り込みデータのコーパスに対して実行され、それ以外のデコーダはすべて `tdstest.py` 内の合成ベクトルに対して検査されます。各デコーダには、`bitecheck.py` 内に対応する変異(mutation bite)があり、意図的にデコーダを壊してテストがそれに気づくことを確認します。プロトコルを追加するたびに、そのテストケースを追加してください。



注意. テストされていないデコードは、いずれ正しく動かなくなります。結果に頼る前に、追加したデコーダごとに合成ベクトルを追加し、そのベクトルが実際に効いていることを証明する変異(mutation bite)を追加してください。

チェックリスト

- 頼る前に
- ファイル名は小文字でスペースを含まず、アンダースコアで始まっていない。
 - クラスが NAME、DESCRIPTION、SOURCES、PARAMS を設定しており、NAME が空でない。
 - `decode()` が、失敗経路を含むすべての経路で (frames, note) のタプルを返す。
 - Frame の index と end_index が絶対値である。すなわち `sig.first_index` にローカルのサンプルインデックスを加えたもの。
 - 読み取れないレコードでは、例外ではなく、空のリストと理由を説明するノートを返す。
 - `columns()`、`row()`、`label()` が、フィールドの数と順序について一致している。
 - 合成テストがデコーダを動かし、デコードした値をアサートする。